

FRED TALKS

19th July 2026

CONTENTS

The Night Watch

JAMES MICKENS · 2675 WORDS

A PM's Guide to AI Agent Architecture: Why Capability Doesn't
Equal Adoption

UMANG · 1871 WORDS

The Night Watch

JAMES MICKENS · 11 APR 2026 · [SOURCE](#)

The Night Watch

JAMES MICKENS



James Mickens is a researcher in the Distributed Systems group at Microsoft's Redmond lab. His current research focuses on web applications,

with an emphasis on the design of JavaScript frameworks that allow developers to diagnose and fix bugs in widely deployed web applications. James also works on fast, scalable storage systems for datacenters.

James received his PhD in computer science from the University of Michigan, and a bachelor's degree in computer science from Georgia Tech. mickens@microsoft.com

s a highly trained academic researcher, I spend a lot of time trying to advance the frontiers of human knowledge. However, as someone who was born in the South, I secretly believe that true progress is

A

a fantasy, and that I need to prepare for the end times, and for the chickens coming home to roost, and fast zombies, and slow zombies, and the polite zombies who say “sir” and “ma’am” but then try to eat your brain to acquire your skills. When the revolution comes, I need to be prepared; thus, in the quiet moments, when I’m not producing incredible scientific breakthroughs, I think about what I’ll do when the weather forecast inevitably becomes

RIVERS OF BLOOD ALL DAY EVERY DAY. The main thing that I ponder is who will be in my gang, because the likelihood of post-apocalyptic survival is directly related to the size and quality of your rag-tag group of associates. There are some obvious people who I'll need to recruit: a locksmith (to open doors); a demolitions expert (for when the locksmith has run out of ideas); and a person who can procure, train, and then throw snakes at my enemies

(because, in a world without hope, snake throwing is a reasonable way to resolve disputes). All of these people will play a role in my ultimate success as a dystopian warlord philosopher. However, the most important person in my gang will be a systems programmer. A person who can debug a device driver or a distributed system is a person who can be trusted in a Hobbesian nightmare of breathtaking scope; a systems programmer has seen the terrors of the world and understood the intrinsic horror of existence. The systems programmer has written drivers for buggy devices whose firmware was implemented by a drunken child or a sober goldfish. The systems programmer has traced a network problem across eight machines, three time zones, and a brief diversion into Amish country, where the problem was transmitted in the front left hoof of a mule named Deliverance. The systems programmer has read the kernel source, to better understand the deep ways of the universe, and the systems programmer has seen the comment in the scheduler that says "DOES THIS WORK LOL," and the systems programmer has wept instead of LOLed, and the systems programmer has submitted a kernel patch to restore balance to The Force and fix the priority inversion that was causing MySQL to hang. A systems programmer will know what to do when society breaks down, because the systems programmer already lives in a world without law.

Listen: I'm not saying that other kinds of computer people

are useless. I believe (but cannot prove) that PHP developers have souls. I think it's great that database people keep trying to improve select-from-where, even though the only queries that cannot be expressed using select-from-where are inappropriate limericks from "The Canterbury Tales." In some way that I don't yet understand, I'm glad that theorists are investigating the equivalence between five-dimensional Turing machines and Edward Scissorhands. In most situations, GUI designers should not be forced to fight each other with tridents and nets as I yell "THERE ARE NO MODAL DATABASE LOGS IN SPARTA." I am like the Statue of Liberty: I accept everyone, even the wretched and the huddled and people who enjoy Haskell. But when things get tough, I need mission-critical people; I need a person who can wear night-vision goggles and descend from a helicopter on ropes and do classified things to protect my freedom while country music plays in the background. A systems person can do that. I can realistically give a kernel hacker a nickname like "Diamondback" or "Zeus Hammer." In contrast, no one has ever said, "These semi-

transparent icons are really semi-transparent! IS THIS THE WORK OF ZEUS HAMMER?”

I picked that last example at random. You must believe me when I say that I have the utmost respect for HCI people.

However, when HCI people debug their code, it's like an

art show or a meeting of the United Nations. There are tea breaks and witticisms exchanged in French; wearing a non- functional scarf is optional, but encouraged. When HCI code doesn't work, the problem can be resolved using grand theories that relate form and perception to your deeply personal feelings about ovals. There will be rich debates about the socioeconomic implications of Helvetica Light, and at some point, you will have to decide whether serifs are daring statements of modernity, or tools of hegemonic oppression that implicitly support feudalism and illiteracy. Is pinching-and- dragging less elegant than circling-and-lightly- caressing?

These urgent mysteries will not solve themselves. And yet, after a long day of debugging HCI code, there is always hope, and there is no true anger; even if you fear that your drop- down list should be a radio button, the drop-down list will suffice until tomorrow, when the sun will rise, glorious and

vibrant, and inspire you to combine scroll bars and left-click- ing in poignant ways that you will commemorate in a sonnet when you return from your local farmer's market.

This is not the world of the systems hacker. When you debug a distributed system or an OS kernel, you do it Texas-style. You gather some mean, stoic people, people who have seen things die, and you get some primitive tools, like a compass and a rucksack and a stick that's pointed on one end, and you walk into the wilderness and you look for trouble, possibly while

using chewing tobacco. As a systems hacker, you must be pre- pared to do savage things, unspeakable things, to kill runaway threads with your bare hands, to write directly to network ports using telnet and an old copy of an RFC that you found in the Vatican. When you debug systems code, there are no high- level debates about font choices and the best kind of turquoise, because this is the Old Testament, an angry and monochro- matic world, and it doesn't matter whether your Arial is Bold or Condensed when people are covered in boils and pestilence and Egyptian pharaoh oppression. HCI people discover bugs by receiving a concerned email from their therapist. Systems people discover bugs by waking up and discovering that their first-born children are missing and “ETIMEDOUT ” has been written in blood on the wall.

What is despair? I have known it—hear my song. Despair is when you’re debugging a kernel driver and you look at a memory dump and you see that a pointer has a value of 7. THERE IS NO HARDWARE ARCHITECTURE THAT IS ALIGNED ON

7. Furthermore, 7 IS TOO SMALL AND ONLY EVIL CODE WOULD TRY TO ACCESS SMALL NUMBER MEMORY.

Misaligned, small-number memory accesses have stolen decades from my life. The only things worse than misaligned, small-number memory accesses are accesses with aligned buffer pointers, but impossibly large buffer lengths. Nothing ruins a Friday at 5 P.M. faster than taking one last pass through the log file and discovering a word-aligned buffer address, but a buffer length of NUMBER OF ELECTRONS IN THE UNI-

VERSE. This is a sorrow that lingers, because a 2893 byte read is the only thing that both Republicans and Democrats agree is wrong. It’s like, maybe Medicare is a good idea, maybe not, but there’s no way to justify reading everything that ever existed a jillion times into a megajillion sized array. This constant war on happiness is what non-systems people do not understand about the systems world. I mean, when a machine learning

algorithm mistakenly identifies a cat as an elephant, this is

actually hilarious. You can print a picture of a cat wearing an elephant costume and add an ironic caption that will entertain people who have middling intellects, and you can hand out copies of the photo at work and rejoice in the fact that everything is still fundamentally okay. There is nothing funny to print when you have a misaligned memory access, because your machine is dead and there are no printers in the spirit world.

An impossibly large buffer error is even worse, because these errors often linger in the background, quietly overwriting your state with evil; if a misaligned memory access is like a criminal burning down your house in a fail-stop manner, an impossibly large buffer error is like a criminal who breaks into your house, sprinkles sand atop random bedsheets and toothbrushes, and then waits for you to slowly discover that your world has been tainted by madness. Indeed, the common discovery mode for an impossibly large buffer error is that your program seems to

be working fine, and then it tries to display a string that should say “Hello world,” but instead it prints “#a[5]:3!” or another syntactically correct Perl script, and you’re like WHAT THE HOW THE, and then you realize that your prodigal memory accesses have been stomping around the heap like the Incredible Hulk when asked to write an essay entitled “Smashing Considered Harmful.”

You might ask, “Why would someone write code in a grotesque language that exposes raw memory addresses? Why not use

a modern language with garbage collection and functional programming and free massages after lunch?” Here’s the answer: Pointers are real. They’re what the hardware understands. Somebody has to deal with them. You can’t just place a LISP book on top of an x86 chip and hope that the hardware learns about lambda calculus by osmosis. Denying the existence of pointers is like living in ancient Greece and denying the existence of Krackens and then being confused about why none of your ships ever make it to Morocco, or Ur-Morocco,

or whatever Morocco was called back then. Pointers are like Krackens—real, living things that must be dealt with so that polite society can exist. Make no mistake, I don’t want to write systems software in a language like C++. Similar to the Necronomicon, a C++ source code file is a wicked, obscure document that’s filled with cryptic incantations and forbidden knowledge. When it’s 3 A.M., and you’ve been debugging for 12 hours, and you encounter a virtual static friend protected volatile

templated function pointer, you want to go into hibernation and awake as a werewolf and then find the people who wrote the C++ standard and bring ruin to the things that they love. The C++ STL, with its dyslexia-inducing syntax blizzard of colons and angle brackets, guarantees that if you try to declare any reasonable data structure, your first seven attempts will result in compiler errors of Wagnerian fierceness:

```
Syntax error: unmatched thing in thing from std::nonstd::map<_Cyrillic, _$$$dollars>const
basic_string< epic_mystery, mongoose_traits &lt; char>, _default_alloc<casual_Fridays =
maybe>>
```

One time I tried to create a `list<map<int>>`, and my syntax errors caused the dead to walk among the living. Such things are clearly unfortunate. Thus, I fully support high-level languages in which pointers are hidden and types are strong and the declaration of data structures does not require you to solve a syntactical puzzle generated by a malevolent extraterrestrial species. That being said, if you find yourself drinking a martini and writing programs in garbage-collected, object-oriented Esperanto, be aware that the only reason that the Esperanto runtime works is because there are systems people who have exchanged any hope of losing their virginity for the exciting opportunity to think about hex numbers and their relationships

with the operating system, the hardware, and ancient blood rituals that Bjarne Stroustrup performed at Stonehenge.

Perhaps the worst thing about being a systems person is that other, non-systems people think that they understand the daily tragedies that compose your life. For example, a few weeks ago, I was debugging a new network file system that my research group created. The bug was inside a kernel-mode component, so my machines were crashing in spectacular and vindic-

tive ways. After a few days of manually rebooting servers, I had transformed into a shambling, broken man, kind of like a computer scientist version of Saddam Hussein when he was pulled from his bunker, all scraggly beard and dead eyes and florid, nonsensical ramblings about semi-imagined enemies.

As I paced the hallways, muttering Nixonian rants about my code, one of my colleagues from the HCI group asked me what my problem was. I described the bug, which involved concurrent threads and corrupted state and asynchronous message delivery across multiple machines, and my coworker said,

“Yeah, that sounds bad. Have you checked the log files for errors?” I said, “Indeed, I would do that if I hadn’t broken every component that a logging system needs to log data. I have a network file system, and I have broken the network, and I have broken the file system, and my machines crash when I make eye contact with them. I HAVE NO TOOLS BECAUSE I’VE DESTROYED MY TOOLS WITH MY TOOLS. My only logging

option is to hire monks to transcribe the subjective experience of watching my machines die as I weep tears of blood.” My co-worker, in an earnest attempt to sympathize, recounted one of his personal debugging stories, a story that essentially involved an addition operation that had been mistakenly replaced with

a multiplication operation. I listened to this story, and I said, “Look, I get it. Multiplication is not addition. This has been known for years. However, multiplication and addition are at least related. Multiplication is like addition, but with more addition. Multiplication is a grown-up pterodactyl, and addition is a baby pterodactyl. Thus, in your debugging story, your code is wayward, but it basically has the right idea. In contrast, there is no family-friendly GRE analogy that relates what my code should do, and what it is actually doing. I had the mod-

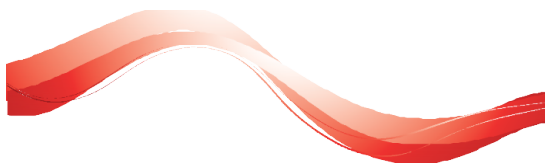
est goal of translating a file read into a network operation, and now my machines have tuberculosis and orifice containment issues. Do you see the difference between our lives? When you asked a girl to the prom, you discovered that her father was a cop. When I asked a girl to the prom, I DISCOVERED THAT HER FATHER WAS STALIN.”

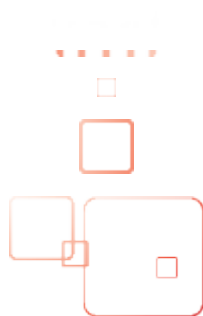
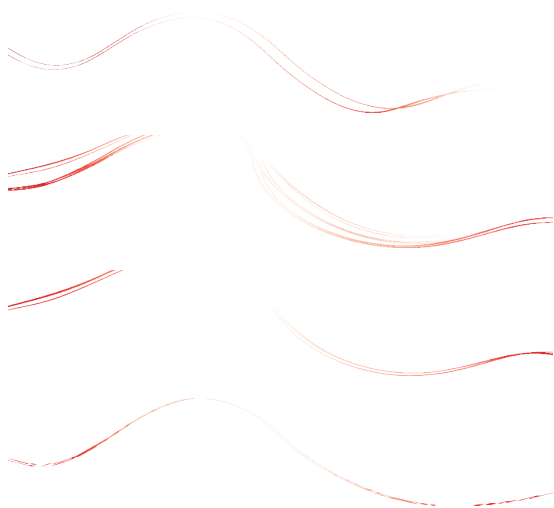
In conclusion, I'm not saying that everyone should be a systems hacker. GUIs are useful. Spell-checkers are useful. I'm glad that people are working on new kinds of bouncing icons because they believe that humanity has solved cancer and homelessness and now lives in a consequence-free world

of immersive sprites. That's exciting, and I wish that I could join those people in the 27th century. But I live here, and I live now, and in my neighborhood, people are dying in the streets. It's like, French is a great idea, but nobody is going to invent French if they're constantly being attacked by bears. Do you see? SYSTEMS HACKERS SOLVE THE BEAR MENACE.

Only through the constant vigilance of my people do you get

the freedom to think about croissants and subtle puns involving the true father of Louis XIV. So, if you see me wandering the halls, trying to explain synchronization bugs to confused monks, rest assured that every day, in every way, it gets a little better. For you, not me. I'll always be furious at the number 7, but such is the hero's journey.







**u
s
e
n
x**

THE ADVANCED
COMPUTING SYSTEMS
ASSOCIATION



Do you know about the USENIX Open Access Policy?

USENIX is the first computing association to offer free and open access to all of our conferences proceedings and videos. We stand by our mission to foster excellence and innovation while supporting research with a practical bias. Your membership fees play a major role in making this endeavor successful.

Please help us support open access.

Renew your USENIX membership

and ask your colleagues to join or renew today!

www.usenix.org/membership

A PM's Guide to AI Agent Architecture: Why Capability Doesn't Equal Adoption

UMANG · 05 SEP 2025 · [SOURCE](#)

Last week, I was talking to a PM who'd in the recent months shipped their AI agent. The metrics looked great: 89% accuracy, sub-second respond times, positive user feedback in surveys. But users were abandoning the agent after their first real problem, like a user with both a billing dispute and a locked account.

"Our agent could handle routine requests perfectly, but when faced with complex issues, users would try once, get frustrated, and immediately ask for a human."

This pattern is observed across every product team that focuses on making their agents "smarter" when the real challenge is making architectural decisions that shape how users experience and begin to trust the agent.

In this post, I'm going to walk you through the different layers of AI agent architecture. How your product decisions determine whether users trust your agent or abandon it. By the end of this, you'll understand why some agents feel "magical" while others feel "frustrating" and more importantly, how PMs should architect for the magical experience.

We'll use a concrete customer support agent example throughout, so you can see exactly how each architectural choice plays out in practice. We'll also see why the counterintuitive approach to trust (hint: it's not about being right more often) actually works better for user adoption.

Let's say you're building a customer support agent

You're the PM building an agent that helps users with account issues - password resets, billing questions, plan changes. Seems straightforward, right?



But when a user says *"I can't access my account and my subscription seems wrong"* what should happen?

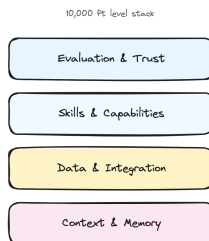
Scenario A: Your agent immediately starts checking systems. It looks up the account, identifies that the password was reset yesterday but the email never arrived, discovers a billing issue that downgraded the plan, explains exactly what happened, and offers to fix both issues with one click.

Scenario B: Your agent asks clarifying questions. "When did you last successfully log in? What error message do you see? Can you tell me more about the subscription issue?" After gathering info, it says "Let me escalate you to a human who can check your account and billing."

Same user request. Same underlying systems. Completely different products.

The Four Layers Where Your Product Decisions Live

Think of agent architecture like a stack where each layer represents a product decision you have to make.



Layer 1: Context & Memory (What does your agent remember?)

The Decision: How much should your agent remember, and for how long?

This isn't just technical storage - it's about creating the illusion of understanding. Your agent's memory determines whether it feels like talking to a robot or a knowledgeable colleague.

For our support agent: Do you store just the current conversation, or the customer's entire support history? Their product usage patterns? Previous complaints?

Types of memory to consider:

- **Session memory:** Current conversation ("You mentioned billing issues earlier...")
- **Customer memory:** Past interactions across sessions ("Last month you had a similar issue with...")
- **Behavioral memory:** Usage patterns ("I notice you typically use our mobile app...")
- **Contextual memory:** Current account state, active subscriptions, recent activity

The more your agent remembers, the more it can anticipate needs rather than just react to questions. Each layer of memory makes responses more intelligent but increases complexity and cost.

The Decision: Which systems should your agent connect to, and what level of access should it have?

The deeper your agent connects to user workflows and existing systems, the harder it becomes for users to switch. This layer determines whether you're a tool or a platform.

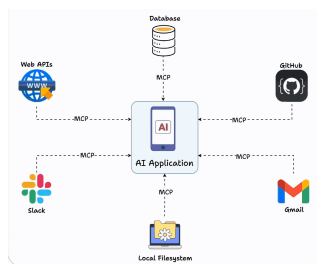
For our support agent: Should it integrate with just your Stripe's billing system, or also your Salesforce CRM, ZenDesk ticketing system, user database, and audit logs? Each integration makes the agent more useful but also creates more potential failure points - *think API rate limits, authentication challenges, and system downtime.*

Layer 3: Skills & Capabilities (What makes you different?)

The Decision: Which specific capabilities should your agent have, and how deep should they go?

Your skills layer is where you win or lose against competitors. It's not about having the most features - it's about having the right capabilities that create user dependency.

For our support agent: Should it only read account information, or should it also modify billing, reset passwords, and change plan settings? Each additional skill increases user value but also increases complexity and risk.



Implementation note: Tools like MCP (Model Context Protocol) are making it much easier to build and share skills across different agents, rather than rebuilding capabilities from scratch.

Layer 4: Evaluation & Trust (How do users know what to expect?)

The Decision: How do you measure success and communicate agent limitations to users?

This layer determines whether users develop confidence in your agent or abandon it after the first mistake. It's not just about being accurate - it's about being trustworthy.

For our support agent: Do you show confidence scores ("I'm 85% confident this will fix your issue")? Do you explain your reasoning ("I checked three systems and found...")? Do you always confirm before taking actions ("Should I reset your password now?")? Each choice affects how users perceive reliability.

Trust strategies to consider:

- **Confidence indicators:** "I'm confident about your account status, but let me double-check the billing details"
- **Reasoning transparency:** "I found two failed login attempts and an expired payment method"
- **Graceful boundaries:** "This looks like a complex billing issue - let me connect you with our billing specialist who has access to more tools"
- **Confirmation patterns:** When to ask permission vs. when to act and explain

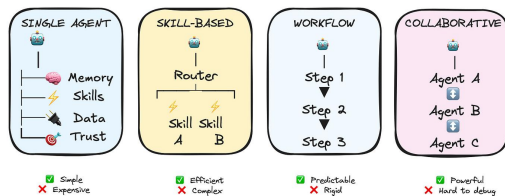
| The counterintuitive insight: users trust agents more when they admit uncertainty than when they confidently make mistakes.

So how do you actually architect an agent?

Okay, so you understand the layers. Now comes the practical question that every PM asks: "How do I actually implement this? How does the agent talk to the skills? How do skills access data? How does evaluation happen while users are waiting?"

Your orchestration choice determines everything about your development experience, your debugging process, and your ability to iterate quickly.

Four Agent Orchestration Patterns



Lets walk through the main approaches, and I'll be honest about when each one works and when it becomes a nightmare.

1. Single-Agent Architecture (Start Here)

Everything happens in one agent's context.

For our support agent: *When the user says "I can't access my account," one agent handles it all - checking account status, identifying billing issues, explaining what happened, offering solutions.*

Why this works: Simple to build, easy to debug, predictable costs. You know exactly what your agent can and can't do.

Why it doesn't: Can get expensive with complex requests since you're loading full context every time. Hard to optimize specific parts.

Most teams start here, and honestly, many never need to move beyond it. If you're debating between this and something more complex, start here.

You have a router that figures out what the user needs, then hands off to specialized skills.

For our support agent: *Router realizes this is an account access issue and routes to the `LoginSkill`. If the LoginSkill discovers it's actually a billing problem, it hands off to `BillingSkill`.*

Real example flow:

1. User: "I can't log in"
2. Router → LoginSkill

3. *LoginSkill checks: Account exists ✓, Password correct ✗, Billing status... wait, subscription expired*

4. *LoginSkill → BillingSkill: "Handle expired subscription for user123"*

5. *BillingSkill handles renewal process*

Why this works: More efficient - you can use cheaper models for simple skills, expensive models for complex reasoning. Each skill can be optimized independently.

Why it doesn't: Coordination between skills gets tricky fast. Who decides when to hand off? How do skills share context?

Here's where MCP really helps - it standardizes how skills expose their capabilities, so your router knows what each skill can do without manually maintaining that mapping.

You predefine step-by-step processes for common scenarios. Think LangGraph, CrewAI, AutoGen, N8N, etc.

For our support agent: *"Account access problem" triggers a workflow:*

1. *Check account status*
2. *If locked, check failed login attempts*
3. *If too many failures, check billing status*
4. *If billing issue, route to payment recovery*
5. *If not billing, route to password reset*

Why this works: Everything is predictable and auditable. Perfect for compliance-heavy industries. Easy to optimize each step.

Why it doesn't: When users have weird edge cases that don't fit your predefined workflows, you're stuck. Feels rigid to users.

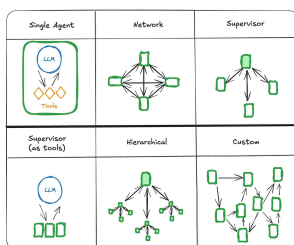
Multiple specialized agents work together using A2A (agent-to-agent) protocols.

The vision: Your agent discovers that another company's agent can help with issues, automatically establishes a secure connection, and collaborates to solve the customer's problem. *Think a booking.com agent interacting with an American Airlines agent!*

For our support agent: ``AuthenticationAgent`` handles login issues, ``BillingAgent`` handles payment problems, ``CommunicationAgent`` manages user interaction. They coordinate through standardized protocols to solve complex problems.

Reality check: This sounds amazing but introduces complexity around security, billing, trust, and reliability that most companies aren't ready for. We're still figuring out the standards.

This can produce amazing results for sophisticated scenarios, but debugging multi-agent conversations is genuinely hard. When something goes wrong, figuring out which agent made the mistake and why is like detective work.



But here's what's interesting - even with the perfect architecture, your agent can still fail if users don't trust it. That brings us to the most counterintuitive lesson about building agents.

The trust thing that everyone gets wrong

Here's something counterintuitive: Users don't trust agents that are right all the time. They trust agents that are honest about when they might be wrong.

Think about it from the user's perspective. Your support agent confidently says "I've reset your password and updated your billing address." User thinks "great!" Then they try to log in and... it doesn't work. Now they don't just have a technical problem - they have a trust problem.

Compare that to an agent that says "I think I found the issue with your account. I'm 80% confident this will fix it. I'm going to reset your password and update your billing address. If this doesn't work, I'll immediately escalate to a human who can dive deeper."

Same technical capability. Completely different user experience.

Building trusted agents requires focus on three things:

1. **Confidence calibration:** When your agent says it's 60% confident, it should be right about 60% of the time. Not 90%, not 30%. Actual 60%.
2. **Reasoning transparency:** Users want to see the agent's work. "I checked your account status (active), billing history (payment failed yesterday), and login attempts (locked after 3 failed attempts). The issue seems to be..."
3. **Graceful escalation:** When your agent hits its limits, how does it hand off? A smooth transition to a human with full context is much better than "I can't help with that."

A lot of times we obsess over making agents more accurate, when what users actually want was more transparency about the agent's limitations.

What's Coming Next

In Part 2, I'll dive deeper into the autonomy decisions that keep most PMs up at night. How much independence should you give your agent? When should it ask for permission vs forgiveness? How do you balance automation with user control?

We'll also walk through the governance concerns that actually matter in practice - not just theoretical security issues, but the real implementation challenges that can make or break your launch timeline.